Chapter 3 — Control Flow: Decisions and Loops

So far every program ran top to bottom, once. **Control flow** is how a program chooses *which* statements run (`if`, `match`) and *how many times* (`for`, `while`). With the truthiness rules from Chapter 2 (how Python decides whether any value counts as true or false — empty and zero values count as false) and the constructs here you can express any algorithm (a step-by-step recipe for solving a problem).

What you'll learn

#	Topic	Key ideas
3.1	Conditional statements — overview	what "branching" means, where it's used
3.2	`if` / `elif` / `else`	syntax, order of tests, nesting vs guard clauses
3.3	Conditional expressions	`a if cond else b`
3.4	`match` / `case`	literal, OR, capture, wildcard, sequence, mapping, class patterns, guards, `as`
3.5	Loops — overview and the iteration protocol	iterables (objects you can loop over), iterators, `for` vs `while`
3.6	`for` loops	lists, strings, dicts, nested loops, `range`, `enumerate`, `zip`
3.7	`while` loops	condition-driven repetition, infinite loops, sentinels (special "stop here" values)
3.8	Loop control	`break`, `continue`, `pass`
3.9	`for-else` / `while-else`	"no break happened"
3.10	Loop idioms	`sum` / `any` / `all`, comprehensions (one-line collection builders), counting, safe modification

Code: `code/conditionals.py`, `code/pattern_matching.py`, `code/loops.py` — outputs are real (CPython 3.14).

3.1 Conditional statements — overview

What it is

A **conditional statement** evaluates a **condition** — any expression, interpreted by its truthiness — and runs a block of code only when the condition holds. Analogy: a railway switch: the train (execution) goes down one track or the other depending on the signal.

Python offers two decision constructs:

- `if` / `elif` / `else` — test conditions one after another.
- `match` / `case` (Python 3.10+) — compare one value against **patterns** describing its shape; a far more powerful cousin of `switch` in C/Java (a statement that jumps to the branch labelled with a matching value).

Why it matters — real-world uses

Domain	Decision
Input validation	Is the email well-formed? Is the password long enough? Reject or accept.
App navigation	Logged in → dashboard; otherwise → login screen.
Games	Health ≤ 0 → game over; score above threshold → next level; difficulty tuned by player behaviour.
Healthcare software	Heart rate outside safe range → alert staff; lab values → suggested treatment protocol.
Web back-ends	Route (send each request to the right handler) by HTTP method (GET to read, POST to submit…) and path; return 404 (the "not found" status code) if not found.

TL;DR

Conditionals choose which code runs; `if` tests conditions in order, `match` compares a value against patterns.

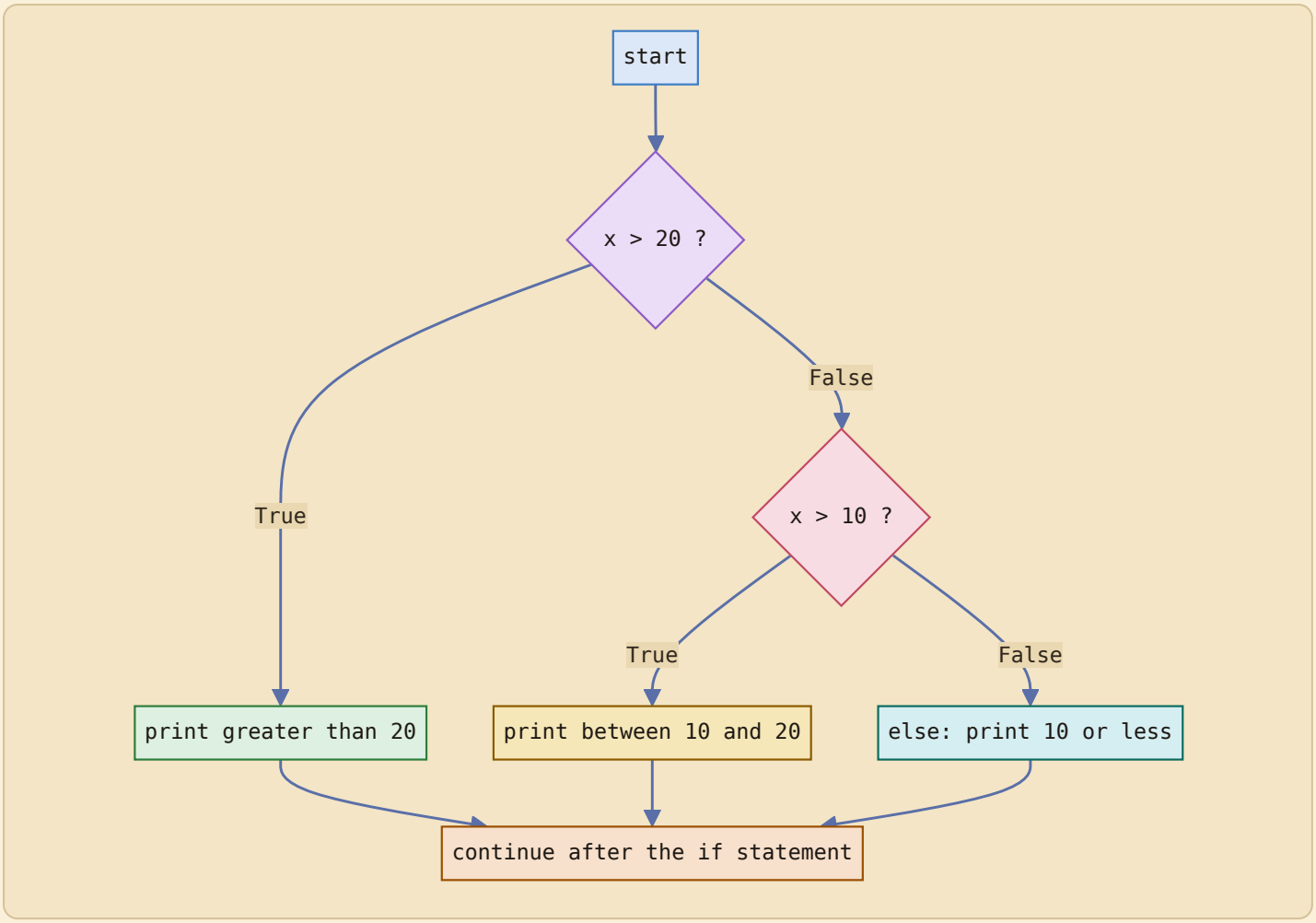
3.2 `if`, `elif`, `else`

What it is

```
if condition1:
    ...            # runs if condition1 is truthy
elif condition2:
    ...            # runs if condition1 was falsy and condition2 is truthy
else:
    ...            # runs if every condition above was falsy
```

- `if` is required; `elif` ("else if") may appear any number of times; `else` is optional and must come last.
- Each header (the line that introduces a block) ends with `:` and its block is indented (Chapter 1).
- **At most one** branch runs: the first whose condition is truthy. The rest are skipped without being evaluated.

How it works



From `code/conditionals.py`:

```
x = 10
if x > 5:
    print("x is greater than 5")          # runs

x = 3
if x > 5:
    print("x is greater than 5")
else:
    print("x is not greater than 5")      # runs

def describe(x: int) -> str:
    if x > 20:
        return "x is greater than 20"
    elif x > 10:
        return "x is greater than 10 but not more than 20"
    else:
        return "x is 10 or less"
```

Output:

```
x is greater than 5
x is not greater than 5
25 -> x is greater than 20
15 -> x is greater than 10 but not more than 20
5 -> x is 10 or less
```

For `x = 15`, the first test (`15 > 20`) fails, the `elif` test (`15 > 10`) passes, and the `else` is never considered.

Order matters

Because the first true branch wins, test the **most specific** condition first:

```
def grade(score: int) -> str:
    if not 0 <= score <= 100:
        raise ValueError(f"score out of range: {score}")
    if score >= 90:
        return "A"
    elif score >= 80:
        return "B"
    elif score >= 70:
        return "C"
    return "F"
```

```
grades: ['A', 'B', 'C', 'F']
```

If `score >= 70` came first, every A and B student would get a C.

Nesting vs guard clauses

You can nest `if` statements, but deep nesting ("arrow code") is hard to read. A **guard clause** handles a special case early and returns, keeping the main path flat — "flat is better than nested" (PEP 20):

```
def shipping_cost(total: float, is_member: bool) -> float:
    if total <= 0:
        return 0.0
    if is_member or total >= 50:
        return 0.0
    return 4.99
```

```
shipping: 4.99 0.0 0.0
```

Writing good conditions

- Rely on truthiness: `if not cart:` rather than `if len(cart) == 0:`.
- Test `None` explicitly when `0` / `""` are valid: the script shows `discount = 0` looks "missing" to a truthiness test but not to `is None`:

```
cart is empty
hello, guest
falsy check says missing: True | is-None check says missing: False
```

- Use chained comparisons for ranges: `if 18 <= temp <= 26 and not raining:`.
- Use `in` instead of long `or` chains: `if day in {"sat", "sun"}:`.

```
weekend
good weather for a walk
```

Trade-offs

Approach	Good for	Weak at
`if/elif/else`	Arbitrary boolean conditions, ranges, few branches	Many equality tests on one value get repetitive
`match/case`	One value, many shapes/values; destructuring (pulling parts out into names)	Plain boolean conditions (use `if`)
dict lookup `{"a": f, "b": g}[key]()`	Many constant keys → action (a *dispatch table*)	Conditions that aren't exact keys

Common mistakes

- `=` instead of `==` in a condition → `SyntaxError` (Python protects you here).
- Missing colon or indentation → `SyntaxError` / `IndentationError`.
- `if x == 1 or 2:` — always true; write `if x in (1, 2):`.
- `else if` → must be `elif`.
- Writing `if flag == True:` → just `if flag:`.
- Overlapping ranges in the wrong order (see grading example).

Real-world examples

- Django views (functions that handle one web request): `if request.method == "POST": … else: …` is the classic form-handling pattern.
- Feature flags (switches that turn a feature on for some users without redeploying): `if flags.is_enabled("new_checkout", user): …`.
- Input validation in CLI tools (command-line interface programs, used by typing commands in a terminal) before converting types.

In the interview

> **30-second answer:** "`if` evaluates conditions top-down by truthiness and runs only the first matching branch; `elif` chains alternatives and `else` catches the rest. I order conditions from most specific to most general, use guard clauses to avoid deep nesting, and `is None` when zero or empty values are legitimate."

- *Does Python have `switch`?* → Not as such; `match` (3.10+) covers it and more, and dicts serve as dispatch tables.
- *Are later `elif` conditions evaluated after one matches?* → No, they're skipped.

Self-check

1. With `x = 20`, which branch of `describe` runs? → The `elif`: `20 > 20` is false, `20 > 10` is true → "x is greater than 10 but not more than 20".
2. Can you have `elif` without `if`? → No.
3. How do you express "between 1 and 10 inclusive"? → `1 <= n <= 10`.
4. What's a guard clause? → An early `return`/`raise` (`raise` signals an error — an *exception*, Chapter 9) that handles an edge case (an unusual or boundary input) so the main logic isn't nested.

TL;DR

First truthy branch wins; order from specific to general; prefer flat guard clauses; use `in`, chained comparisons and truthiness for readable conditions.

3.3 Conditional expressions (the "ternary")

What it is

An **expression** (produces a value) version of `if/else` — nicknamed the "ternary" because it has three parts:

```
value_if_true if condition else value_if_false
```

```
parity = "even" if n % 2 == 0 else "odd"          # 7 is odd
print("Access granted." if has_access else "Access denied.")
```

Only the chosen branch is evaluated. It has very low precedence (just above `lambda`, the one-line anonymous function), so wrap it in parentheses inside bigger expressions.

When not to use it

- When branches are long or have side effects (effects beyond producing a value, such as printing or changing data) — use a normal `if` statement.
- Nested ternaries (`a if x else b if y else c`) are legal but hard to read.
- For defaults, `x or default` is shorter — but only when falsy values of `x` should be replaced.

Self-check

1. Write `abs` with a conditional expression. → `x if x >= 0 else -x`.
2. Is `else` optional in a conditional expression? → No, it's required.

TL;DR

`a if cond else b` picks a value; keep it short and un-nested.

3.4 `match` / `case` — structural pattern matching

What it is

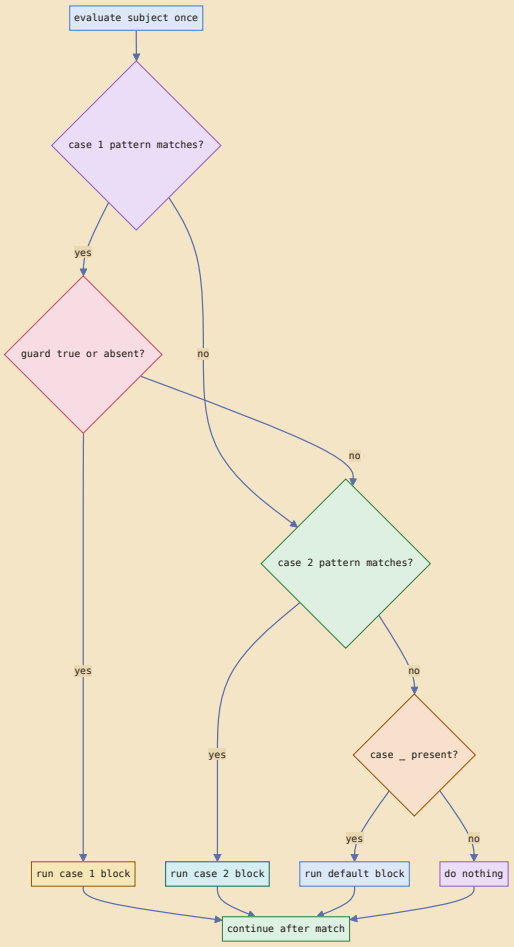
`match` (Python 3.10, PEPs 634–636) takes a **subject** value and tries each `case` **pattern** in order. The first pattern that *matches* (and whose optional guard is true) runs its block; the others are skipped. If none match, nothing happens — no error.

It looks like `switch` from C/Java, but a pattern can describe the **structure** of the data — "a list whose first item is `"go"` followed by one more item", "a dict with a `type` key equal to `"click"`", "a `Point` with `y == 0`" — and **bind** parts of it to names in one step (*destructuring*).

```
match subject:
    case pattern1:
        ...
    case pattern2 if guard:
        ...
    case _:
        ...          # wildcard: the default
```

`match` and `case` are **soft keywords** (special only in this position) — you can still name a variable `match`.

How it works



There is **no fall-through** (in C, a case without `break` keeps running into the next case's code): unlike C's `switch`, you never write `break`.

The pattern kinds

Pattern	Looks like	Matches when…	Binds
Literal	`200`, `"Monday"`, `None`, `True`	subject `==` literal (`None`/`True`/`False` compared with `is`)	—
Capture	`x` (a bare name)	always	subject → `x`
Wildcard	`_`	always	nothing
Value	`Color.RED`, `http.HTTPStatus.OK` (dotted name)	subject `==` that value	—
OR	`p1 | p2 | p3`	any alternative matches	each alternative must bind the same names
AS	`pattern as name`	pattern matches	whole matched part → `name`
Sequence	`[a, b]`, `(a, b)`, `[first, *rest]`	subject is a sequence (list, tuple, …, **not** `str`/`bytes`) of matching length and items	items
Mapping	`{"type": "click", "pos": p}`	subject is a mapping containing those keys with matching values; **extra keys are ignored**	values; `**rest` collects the other items
Class	`Point(x=0, y=0)`, `Point(x, 0)`, `int(n)`, `str()`	`isinstance(subject, Point)` and the listed attributes (values stored on the object) match	attributes
Guard	`case p if cond:`	pattern matches **and** `cond` is truthy	(uses pattern's bindings)

Literal and OR patterns

From `code/pattern_matching.py`:

```
def http_status(code: int) -> str:
    match code:
        case 200:
            return "Success"
        case 404:
            return "Not Found"
        case 500:
            return "Server Error"
        case 301 | 302 | 307 | 308:          # OR pattern
            return "Redirect"
        case _:                              # wildcard
            return "Unknown status code"

def day_kind(day: str) -> str:
    match day:
        case "Monday" | "Friday":
            return "Start/end of the work week"
        case "Tuesday" | "Wednesday" | "Thursday":
            return "Midweek"
        case _:
            return "Weekend"
```

```
['Success', 'Not Found', 'Server Error', 'Redirect', 'Unknown status code']
Midweek | Start/end of the work week | Weekend
```

With `code = 404`: the subject is compared to `200` (no), then `404` (yes) → "Not Found"; later cases are skipped.

Sequence patterns

```python
def first_color(colors: list[str]) -> str:
    match colors:
        case []:
            return "empty list"
        case ["red", *others]:        # first item "red"; others = the rest (a list)
            return f"red first, {len(others)} others"
        case ["blue", *_]:            # *_ = any number of remaining items, not bound
            return "blue first"
        case [single]:                # exactly one item
            return f"only one color: {single}"
        case _:
            return "different first color"
```

```
[] -> empty list
['red', 'blue', 'green'] -> red first, 2 others
['blue', 'red'] -> blue first
['pink'] -> only one color: pink
['pink', 'red'] -> different first color
```

Without a star, the length must match exactly. At most one starred item per sequence pattern.

Sequence patterns shine for parsing commands. Here with an OR sub-pattern captured by `as`, and a **guard**:

```python
def command(text: str) -> str:
    match text.split():
        case ["go", ("north" | "south" | "east" | "west") as direction]:
            return f"moving {direction}"
        case ["take", item]:
            return f"picked up {item}"
        case ["drop", *items] if items:          # guard: need at least one item
            return f"dropped {', '.join(items)}"
        case ["quit" | "exit"]:
            return "bye"
        case _:
            return f"unknown command: {text!r}"
```

```
'go north'       -> moving north
'take lamp'      -> picked up lamp
'drop key coin'  -> dropped key, coin
'drop'           -> unknown command: 'drop'
'quit'           -> bye
'dance'          -> unknown command: 'dance'
```

`"drop"` alone matches the pattern `["drop", *items]` with `items == []`, but the guard `if items` is false, so matching continues to `_`.

Tuples work the same way. Here the guard adds a numeric condition:

```python
match person:                                  # person = (25, "Engineer")
    case (age, "Engineer") if age > 20:
        return f"Adult Engineer aged {age}"
    case (age, profession):
        return f"{profession} aged {age}"
```

```
Adult Engineer aged 25 | Engineer aged 19
```

Strings are not sequences for matching (a deliberate design choice, so `"ab"` doesn't match `[a, b]`):

```
'ab' is matched as a str, not as a sequence of characters
```

Mapping patterns

Ideal for JSON-like dicts (JSON is the text data format most web services exchange), e.g. events or API payloads (the data body sent to or from a web service):

```
match event:
    case {"type": "click", "pos": (x, y)}:          # nested sequence pattern
        return f"click at {x},{y}"
    case {"type": "key", "key": str(k)} if len(k) == 1:
        return f"key {k!r}"
    case {"type": "resize", **rest}:                # rest = remaining key/values
        return f"resize with {rest}"
    case {"type": other}:
        return f"unhandled event type {other!r}"
    case _:
        return "not an event"
```

```
{'type': 'click', 'pos': (3, 4), 'button': 1} -> click at 3,4
{'type': 'key', 'key': 'q'} -> key 'q'
{'type': 'resize', 'w': 800, 'h': 600} -> resize with {'w': 800, 'h': 600}
{'type': 'scroll'} -> unhandled event type 'scroll'
['click'] -> not an event
```

The first event has an extra `"button"` key and still matches — mapping patterns check only the keys you list.

Class patterns

> **Key term — class:** a *class* is a blueprint for creating objects of your own type (Chapter 10); `Point` below is a class and `Point(1, 2)` is one object (an *instance*) of it. `isinstance(obj, Point)` asks "was `obj` made from `Point` (or a subclass)?".

`ClassName(...)` first checks `isinstance`, then matches attributes. Keyword form `Point(x=0)` matches attribute `x`. Positional form `Point(a, b)` needs the class to define `__match_args__` — a tuple naming which attributes the positions map to. `@dataclass` (a decorator — a line starting with `@` that adds behaviour to the definition below it — that auto-writes common class code) generates it automatically (`('x', 'y')`). Built-in types like `int`, `str`, `float` accept one positional sub-pattern that matches the whole subject.

```
@dataclass
class Point:
    x: float
    y: float

match shape:
    case Point(x=0, y=0):                           # keyword attributes
        return "the origin"
    case Point(x, 0):                               # positional via __match_args__
        return f"on the x-axis at {x}"
    case Point():                                   # any Point
        return "some other point"
    case Circle(center=Point(x=0, y=0), radius=r):  # nested patterns
        return f"circle of radius {r} at the origin"
    case Circle(radius=r) if r <= 0:
        return "invalid circle"
    case int(n) | float(n):
        return f"a number {n}"
    case str() as s:
        return f"a string of length {len(s)}"
    case _:
        return "unknown shape"
```

```
Point(x=0, y=0) -> the origin
Point(x=5, y=0) -> on the x-axis at 5
Point(x=1, y=2) -> some other point
Circle(center=Point(x=0, y=0), radius=2) -> circle of radius 2 at the origin
Circle(center=Point(x=1, y=1), radius=-1) -> invalid circle
7 -> a number 7
2.5 -> a number 2.5
'hello' -> a string of length 5
[1] -> unknown shape
Point.__match_args__ = ('x', 'y')
```

The capture-pattern trap

A **bare name** in a pattern is a *capture*, never a comparison. With `RED = "red"`, `case RED:` does **not** mean "equals `RED`" — it matches anything and rebinds `RED`:

```
matched?! blue
SyntaxError: name capture 'RED' makes remaining patterns unreachable
```

Python refuses to compile such a capture if more cases follow it. To compare against a constant, use a **dotted name** (value pattern) such as an `Enum` member (a named constant inside an *enumeration* class):

```python
class Color(Enum):
    RED = "red"
    GREEN = "green"

match c:
    case Color.RED:      # compared with ==
        return "stop"
    case Color.GREEN:
        return "go"
```

```
stop go
```

(or a literal, or a guard: `case x if x == RED:`).

Trade-offs

	match	if/elif
Checks structure (length, keys, type) and extracts parts at once	✅	Manual `isinstance`, `len`, indexing
Arbitrary boolean conditions	Via guards only	✅ natural
Python version	3.10+	All
Performance	Similar — cases are tried in order, not a jump table (a direct lookup of the matching branch)	Similar

When not to use it

- For one or two simple boolean tests — `if` is clearer.
- To map constant keys to values — a dict is simpler (`STATUS = {200: "Success", …}`).
- In code that must run on Python ≤ 3.9.

Common mistakes

- `case RED:` with a plain variable — a capture, not a comparison (above).
- Expecting fall-through or writing `break` inside a case.
- Expecting `case [a, b]:` to match a string.
- Forgetting `case _:` and silently doing nothing for unexpected input — consider raising an error in the default.

- OR patterns binding different names (`case [x] | [x, y]:`) → `SyntaxError` ("alternative patterns bind different names").

Real-world examples

- Command-line and chat-bot command parsers (like `command()` above).
- Handling JSON messages from WebSockets (always-open, two-way browser–server connections) or message queues (services that hold messages until a worker processes them; see System Design Fundamentals) by their `type` field.
- Compilers/interpreters walking an AST (Abstract Syntax Tree, the tree structure a compiler builds from source code): `case BinOp(left, Add(), right):` — the `ast` module's node classes define `__match_args__` for exactly this.
- Handling HTTP status codes and redirect families.

In the interview

> **30-second answer:** "`match` evaluates a subject once and tries case patterns top to bottom, with no fall-through. Patterns can be literals, captures, the `_` wildcard, dotted-name constants, OR-patterns, sequences with `*rest`, mappings that ignore extra keys, and class patterns that check `isinstance` and attributes via `__match_args__`. Guards add an `if` condition. The main trap is that a bare name captures instead of comparing."

- *How do you match a constant?* → Use a dotted name (enum member, module constant), a literal, or a guard.
- *Do mapping patterns require exact keys?* → No, extra keys are ignored; use `**rest` to capture them.
- *What makes positional class patterns work?* → The class's `__match_args__`, auto-generated by `dataclass` and `namedtuple` (a tuple whose fields also have names).

Self-check

1. Does `case {"a": 1}:` match `{"a": 1, "b": 2}`? → Yes.
2. What does `case [x, *_]:` match? → Any non-string sequence with at least one item; binds the first to `x`.
3. What happens when no case matches and there's no `_`? → Nothing; execution continues after the `match`.
4. How do you capture the alternative that matched in `"a" | "b"`? → `("a" | "b") as choice`.
5. Why does `case Point(x, y)` need `__match_args__`? → To know which attributes positions 1 and 2 refer to.

TL;DR

`match` = compare one value against shape-aware patterns, first match wins, no fall-through; use literals, `|`, `[... *rest]`, `{key: ...}`, `Class(attr=...)`, guards and `as`; use dotted names for constants — bare names capture.

3.5 Loops — overview and the iteration protocol

What it is

A **loop** repeats a block of code. Python has two:

Loop	Repeats…	Use when
`for`	once per item of an **iterable**	you have a collection or a known count
`while`	as long as a **condition** stays truthy	you don't know how many repetitions in advance

Loop control statements change the normal flow: `break` (leave the loop), `continue` (skip to the next iteration), `pass` (do nothing — a placeholder). Visiting every item of a collection is called **traversal**.

How `for` really works — the iteration protocol

An **iterable** is any object that can hand out its items one at a time: lists, tuples, strings, dicts, sets, ranges, files, generators (objects that produce values one at a time, on demand)… An **iterator** is the object that keeps track of the position.

1. `for x in obj:` calls `iter(obj)` to get an iterator.
2. Each round, it calls `next(iterator)` and binds the result to `x`.
3. When `next` raises (signals) the `StopIteration` exception, the loop ends (quietly).

```
it = iter(["a", "b"])
next(it)   # 'a'
next(it)   # 'b'
next(it)   # raises StopIteration
```

That is why one `for` statement works for every container, and why lazy (compute each item only when asked) objects like `range` and file handles (the objects `open()` returns) can be looped over without loading everything into memory.

TL;DR

`for` = "for each item", driven by `iter()`/`next()`; `while` = "while condition". Control them with `break`, `continue`, `pass`.

3.6 The `for` loop

What it is

```
for variable in iterable:
    ...   # body runs once per item; variable holds the current item
```

How it works — basic traversal

From `code/loops.py`:

```
for fruit in ["apple", "banana", "cherry"]:
    print(fruit, end=" ")
for ch in "hey":
    print(ch.upper(), end="")
```

```
apple banana cherry
HEY
```

No index bookkeeping, so no off-by-one errors (being wrong by exactly one, e.g. stopping an item early). The loop variable **survives** after the loop with its last value (a quirk to remember).

Looping through dictionaries

Iterating a dict directly yields its **keys** (in insertion order). Use views (live, read-only windows onto the dict's keys, values or pairs) for the rest:

You need	Write
keys	`for k in d:` (or `d.keys()`)
values	`for v in d.values():`
both	`for k, v in d.items():`

```
info = {"name": "John", "age": 30, "city": "New York"}
for key in info:
    print(key, "->", info[key])
for key, value in info.items():          # preferred: no second lookup
    print(f"{key}={value}", end=" ")
```

```
name -> John
age -> 30
city -> New York
name=John age=30 city=New York | values: ['John', 30, 'New York']
```

Nested loops

Every time the outer loop advances by one item, the inner loop runs from start to finish, so the body runs `rows × cols` times.

```
matrix = [[1, 2, 3], [4, 5, 6]]
for row in matrix:
    for num in row:
        print(num, end=" ")
    print()                                # newline after each row
```

```
1 2 3
4 5 6
```

Nested loops multiply work: two loops over n items is O(n²) (Big-O notation: the work grows with the square of the size — 10× the data means 100× the work). If the inner loop only searches for something, a set or dict lookup often removes it.

`range()` — counting loops

`range` produces integers lazily, stop **exclusive**:

Call	Produces
`range(stop)`	0, 1, …, stop−1
`range(start, stop)`	start, …, stop−1
`range(start, stop, step)`	start, start+step, … while < stop (or > stop for negative step)

```
[0, 1, 2, 3, 4, 5, 6, 7, 8, 9] [1, 2, 3, 4, 5, 6, 7, 8, 9, 10] [1, 3, 5, 7, 9] [10, 7, 4, 1]
```

`range(1, 11, 2)` gives the odd numbers 1–9; a negative step counts down. `step` can't be 0 (`ValueError`). `range` supports `len()`, indexing and fast `in`.

`enumerate()` — index and item together

Instead of `for i in range(len(names)): names[i]`, write:

```
for i, name in enumerate(names, start=1):
    print(f"{i}. {name}", end="  ")
```

```
1. ann  2. bob  3. cy
```

`zip()` — walk several iterables in parallel

`zip(a, b)` yields tuples `(a[0], b[0]), (a[1], b[1]), …` and **stops at the shortest** input. Pass `strict=True` (3.10+) to raise an error if lengths differ — usually what you want when the sequences are supposed to align.

```
for name, score in zip(names, scores, strict=True):
    print(f"{name}:{score}", end="  ")
```

```
ann:91  bob:78  cy:85
zip strict: zip() argument 2 is shorter than argument 1
plain zip stops at the shortest: [(1, 'a'), (2, 'b')]
```

`dict(zip(keys, values))` builds a dict from two lists.

Common mistakes

- `for i in range(len(items)):` when you only need the items (use `for item in items`) or both (use `enumerate`).
- Off-by-one with `range`: `range(1, 10)` stops at 9.
- Modifying a list while iterating over it (see §3.10).
- Expecting iteration over a dict to give pairs — it gives keys.
- Silent truncation with plain `zip` on mismatched data.

Real-world examples

- Processing rows from `csv.reader(f)` (CSV = comma-separated values, a plain-text table format) or lines of a file: `for line in f:` streams the file lazily.
- `for i, batch in enumerate(dataloader):` is the standard PyTorch training loop (the `dataloader` hands out the training data in small groups called *batches*).
- `zip(features, labels)` (inputs and their correct answers) pairs data in ML pipelines (chains of machine-learning processing steps).

In the interview

30-second answer: "A `for` loop calls `iter()` on the iterable and `next()` until `StopIteration`, so it works on anything iterable. I iterate items directly, use `enumerate` when I need the index, `zip(strict=True)` for parallel sequences, `.items()` for dicts, and `range` for counts."

- *Why is `range(10**9)` cheap?* → It's lazy — computes values on demand.
- *What does `zip` do with unequal lengths?* → Stops at the shortest; `strict=True` raises `ValueError`.
- *Is the loop variable available after the loop?* → Yes, it keeps the last value (unless the iterable was empty).

Self-check

1. `list(range(2, 11, 3))`? → `[2, 5, 8]`.
2. How do you loop with index starting at 1? → `enumerate(seq, start=1)`.
3. How do you get keys and values from a dict? → `for k, v in d.items():`.
4. How many times does the inner body of two nested loops over 3 and 4 items run? → 12.

TL;DR

`for item in iterable`; dict → keys (use `.items()`); `range(start, stop, step)` excludes stop; `enumerate` for indices; `zip(strict=True)` for parallel lists.

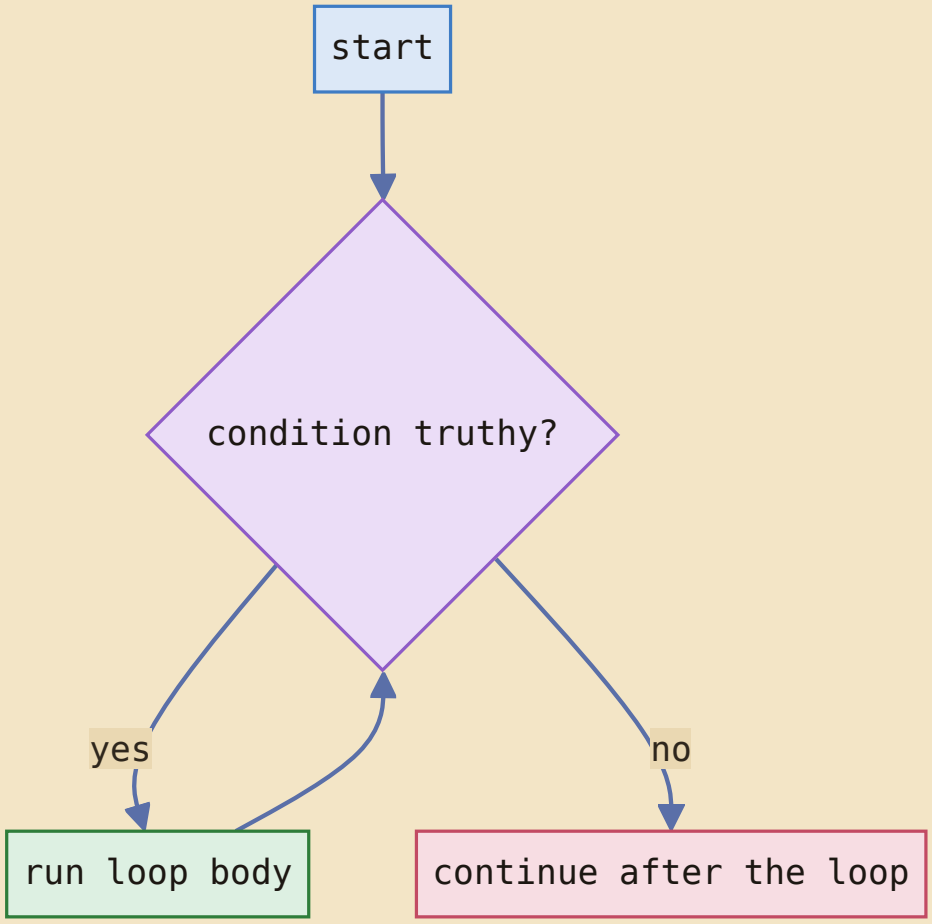
3.7 The `while` loop

What it is

```
while condition:
    ...   # repeats while condition is truthy; re-checked before every iteration
```

Use `while` when the number of iterations isn't known up front: reading until the user types "quit", retrying until success, iterating until a value converges (settles on a final value).

How it works



1. Check the condition. If falsy, skip past the loop (the body may run **zero** times).
2. Run the body.
3. Go back to 1.

The body **must** change something the condition depends on, or the loop never ends.

```
count = 0
while count < 5:
    print(f"Count is {count}", end="; ")
    count += 1          # progress toward making the condition false
```

```
Count is 0; Count is 1; Count is 2; Count is 3; Count is 4;
```

An example where the count is genuinely unknown in advance — how many halvings until a number drops below 1?

```
n, steps = 100.0, 0
while n >= 1:
    n /= 2
    steps += 1
```

```
halvings until < 1: 7
```

(100 → 50 → 25 → 12.5 → 6.25 → 3.125 → 1.5625 → 0.78125.)

Infinite loops on purpose

`while True:` with a `break` inside is the idiomatic "loop and a half" — e.g. a menu or a server loop (a loop that waits for and answers requests forever):

```
while True:
    answer = input("> ")
    if answer == "quit":
        break
    handle(answer)
```

With the walrus operator this can be `while (answer := input("> ")) != "quit": handle(answer)`.

Trade-offs

	for	while
Termination	Automatic when the iterable is exhausted	Your responsibility
Typical bug	Off-by-one in `range`	Infinite loop (forgot to update)
Use for	Collections, fixed counts	Unknown counts, event loops (wait for events, handle them, repeat), retries, convergence

Common mistakes

- Forgetting to update the loop variable → infinite loop (stop it with Ctrl-C, which raises `KeyboardInterrupt`).
- Using `while i < len(items): … i += 1` to traverse a list — use `for`.
- Floating-point conditions like `while x != 1.0:` that never become exactly true — use `<` / `>` or a tolerance.

Real-world examples

- Game loops: `while running: handle_events(); update(); draw()` (Pygame).
- Retry with back-off (waiting longer before each new attempt): `while attempts < 5 and not ok: …`.
- Reading a socket (one end of a network connection) or file in chunks: `while (chunk := f.read(8192)): …`.

In the interview

30-second answer: "`while` re-checks its condition before each iteration and is for when the iteration count is unknown. The body must make progress toward the condition becoming false; for intentional infinite loops I use `while True` with a clear `break`."

Self-check

1. Can a `while` body run zero times? → Yes, if the condition is falsy at the start.
2. What key stops a runaway loop in the terminal? → Ctrl-C (`KeyboardInterrupt`).
3. Rewrite `i = 0; while i < 3: print(i); i += 1` as a `for`. → `for i in range(3): print(i)`.

TL;DR

`while cond:` repeats until `cond` is falsy; guarantee progress; `while True` + `break` for loop-and-a-half.

3.8 Loop control: `break`, `continue`, `pass`

What it is

Statement	Effect
`break`	Exit the **innermost** enclosing loop immediately; skips any loop `else`
`continue`	Skip the rest of this iteration; go to the next one (for `while`, the condition is re-checked)
`pass`	Do nothing; fills a place where syntax requires a statement

How it works

```
for number in [1, 2, 3, 4, 5, 6]:
    if number == 4:
        print("Number 4 found, stopping loop.")
        break                                   # 5 and 6 are never visited

odds = []
for number in [1, 2, 3, 4, 5, 6]:
    if number % 2 == 0:
        continue                                # skip evens
    odds.append(number)

for number in [1, 2, 3]:
    if number % 2 == 0:
        pass                                    # placeholder - does nothing
    print(number, end=" ")                      # still runs for every number
```

```
Number 4 found, stopping loop.
continue kept odds: [1, 3, 5]
1 2 3
```

`pass` is **not** `continue`: after `pass`, the rest of the body still executes. Use `pass` for empty stubs (placeholders to be filled in later) — `def todo(): pass`, `class MyError(Exception): pass` — or `...` (Ellipsis), which works the same way in bodies.

`break` exits only one level:

```
for i in range(3):
    for j in range(3):
        if j == 1:
            break                   # leaves the inner loop only
        print(f"({i},{j})", end=" ")
```

```
(0,0) (1,0) (2,0)
```

To exit several levels, move the loops into a function and `return`, or use a flag (a boolean variable that records that something happened).

Common mistakes

- Expecting `break` to leave all nested loops.
- Using `pass` where `continue` was meant (or vice versa).
- `continue` in a `while` loop placed *before* the counter update → infinite loop.
- Over-using `continue` for filtering when a comprehension (`[x for x in xs if cond]`) is clearer.

In the interview

> **30-second answer:** "`break` leaves the innermost loop, `continue` jumps to the next iteration, `pass` is a no-op placeholder. To break out of nested loops I refactor (restructure without changing behaviour) into a function and `return`."

Self-check

1. Does `pass` skip the rest of the loop body? → No.
2. In a `while` loop, what happens right after `continue`? → The condition is re-evaluated.
3. How do you exit two nested loops at once? → `return` from a function (or a flag / exception).

TL;DR

`break` = leave loop, `continue` = next iteration, `pass` = do nothing. All act on the innermost loop.

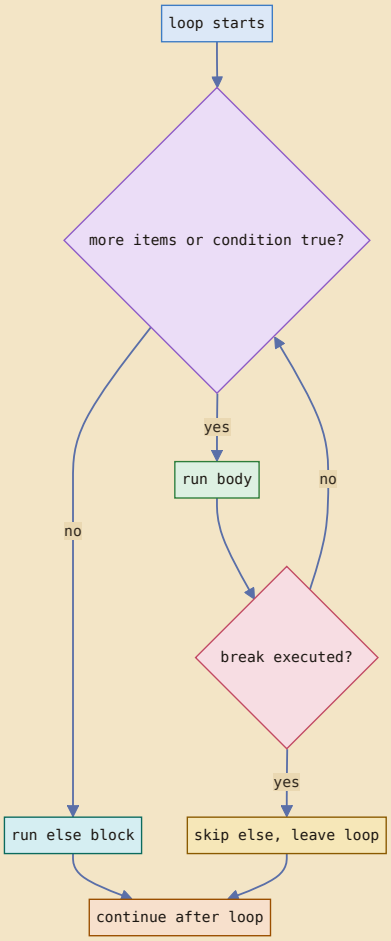
3.9 `for-else` and `while-else`

What it is

Loops can have an `else` block that runs **only if the loop finished without hitting** `break` — the iterable was exhausted (`for`) or the condition became false (`while`). Read it as **"no break"**. Its classic use: search loops, where `break` means "found" and `else` means "not found" — no flag variable needed.

```
for item in iterable:
    ...
else:
    ...   # runs only if the loop didn't break
```

How it works



From `code/loops.py`:

```
for number in [1, 2, 3, 4, 5]:
    if number == 3:
        print("Number 3 found, stopping search.")
        break
else:
    print("Number not found.")        # skipped: break happened

for number in [1, 2, 4, 5]:
    if number == 3:
        print("found 3")
        break
else:
    print("3 not in list (else ran: no break)")

count = 0
while count < 5:
    if count == 2:
        print("Breaking at count 2.")
        break
    count += 1
else:
    print("Loop was uninterrupted.")  # skipped: break happened
```

```
Number 3 found, stopping search.
3 not in list (else ran: no break)
Breaking at count 2.
```

Without any `break` statement in the body, the `else` always runs — e.g. "All numbers processed." after a plain processing loop — which is legal but pointless (just put the code after the loop).

A practical search — primality by trial division (testing whether n is prime by trying every possible divisor up to √n):

```python
def is_prime(n: int) -> bool:
    if n < 2:
        return False
    for d in range(2, int(n ** 0.5) + 1):
        if n % d == 0:
            return False
    return True
```

```
is 29 prime? True | is 21 prime? False
```

(Inside a function, `return` plays the role of `break`, and the final `return True` plays the role of `else`. At module level (the top level of a file, outside any function) or when you need to continue afterwards, `for-else` is the flag-free equivalent.)

Trade-offs / when not to use it

Many developers misread `else` as "if the loop didn't run". If your team finds it confusing, a helper function with `return`, or `any()`/`next()` (§3.10), is often clearer. Always add a comment like `# no break` when you use it.

Common mistakes

- Thinking `else` runs when the loop body never executed only — it also runs after a normal full run.
- Thinking `else` runs after `break` — it doesn't.
- Indenting the `else` to pair with an `if` inside the loop by accident (or vice versa) — alignment decides which statement it belongs to.

In the interview

30-second answer: "A loop's `else` runs only when the loop wasn't terminated by `break` — think 'no break'. It's handy for search loops: `break` on found, `else` for not-found, without a flag variable."

- *Does `else` run if the iterable is empty?* → Yes, there was no break.
- *Does `return` inside the loop skip `else`?* → Yes, it leaves the function entirely.

Self-check

1. `for x in []: pass` `else: print("E")` — prints? → `E`.
2. Is `while-else` `else` run after `break`? → No.
3. What's the mnemonic? → "else = no break".

TL;DR

Loop `else` runs when the loop ends naturally, not via `break`. Use it for search-with-not-found; comment it.

3.10 Common loop idioms

Aggregation with built-ins

Many loops are just a built-in in disguise — shorter, faster (implemented in C) and clearer:

```
nums = [4, -1, 7, 0, 3]
sum(nums), min(nums), max(nums)
any(x < 0 for x in nums)                  # True if at least one; stops early
all(x < 10 for x in nums)                 # True if every one; stops early
[x * x for x in nums if x > 0]            # list comprehension: map + filter
list(reversed(nums)), sorted(nums, reverse=True)
next((x for x in nums if x > 5), None)    # first match or a default
```

```
sum/min/max: 13 -1 7
any negative? True | all < 10? True
squares of positives: [16, 49, 9]
reversed: [3, 0, 7, -1, 4] | sorted desc: [7, 4, 3, 0, -1]
first > 5: 7
```

`next(generator, default)` replaces a whole `for … break … else` search when you just need the first match. Comprehensions get a full section in the lists chapter.

Counting / grouping with a dict

```
counts = {}
for word in "the cat and the hat".split():
    counts[word] = counts.get(word, 0) + 1
```

```
word counts: {'the': 2, 'cat': 1, 'and': 1, 'hat': 1}
```

(`collections.Counter` does this in one call; covered with dictionaries.)

Don't change a collection while looping over it

Removing items from a list during iteration shifts the remaining items left, so the loop **skips** the next one:

```
buggy = [1, 2, 2, 3]
for x in buggy:
    if x == 2:
        buggy.remove(x)        # the second 2 slides into the visited slot and is skipped
```

```
remove-while-iterating left: [1, 2, 3] | filtered correctly: [1, 3]
RuntimeError: dictionary changed size during iteration
```

Fixes: build a new list (`[x for x in data if x != 2]`), or iterate over a copy (`for x in data[:]:`). For dicts, Python refuses outright with `RuntimeError`; iterate over `list(d)` if you must add/delete keys.

Idiom cheat-sheet

Instead of…	Write…
`for i in range(len(xs)): print(xs[i])`	`for x in xs: print(x)`
index + item via `range(len())`	`for i, x in enumerate(xs):`
parallel indexing `a[i], b[i]`	`for x, y in zip(a, b, strict=True):`
`for k in d: v = d[k]`	`for k, v in d.items():`
looping backwards with indices	`for x in reversed(xs):`
accumulator loop (keeps a running total) for total	`sum(xs)`
flag + loop to test a condition	`any(...)` / `all(...)`
loop with `append` to transform	list comprehension
`while` loop reading chunks	`while (chunk := f.read(n)):`

Real-world examples

- Log analysis scripts: `for line in open(path): if "ERROR" in line: …`.
- ETL jobs (Extract–Transform–Load: batch jobs that pull data, reshape it and load it elsewhere) aggregating per-key totals with dicts before loading to a database.
- Validation: `if not all(field in payload for field in REQUIRED): return 400` (the HTTP "bad request" status).

In the interview

> **30-second answer:** "I replace hand-written loops with built-ins where possible — `sum`, `any`, `all`, `min` / `max`, `sorted`, comprehensions, `next` with a default — because they're clearer and run in C. I never mutate a list or dict while iterating it; I build a new one or iterate a copy."

- *Why does removing during iteration skip items?* → The list iterator advances by index; removal shifts later items down one slot.
- *Do `any` / `all` evaluate everything?* → No, they short-circuit.

Self-check

1. `all([])`? `any([])`? → `True`, `False`.
2. Safe way to delete items matching a condition? → Rebuild: `xs = [x for x in xs if not cond(x)]`.
3. First even number or `None` in one line? → `next((x for x in xs if x % 2 == 0), None)`.

TL;DR

Prefer built-ins and comprehensions over manual loops; `enumerate` / `zip` / `.items()` over indexes; never mutate what you're iterating.

Chapter 3 TL;DR

- `if/elif/else`: first truthy branch wins; order from specific to general; guard clauses keep code flat; `a if c else b` for values.
- `match/case` (3.10+): one subject vs patterns — literals, `|`, captures, `_`, dotted constants, sequences with `*rest`, mappings (extra keys ignored), class patterns (`__match_args__`), guards, `as`; no fall-through; bare names capture.
- `for` loops use the iteration protocol (`iter` / `next` / `StopIteration`) and work on any iterable; dicts yield keys; use `range`, `enumerate`, `zip(strict=True)`, `.items()`.
- `while` loops run while a condition holds — guarantee progress; `while True` + `break` for loop-and-a-half.
- `break` leaves the innermost loop, `continue` skips to the next iteration, `pass` does nothing.
- Loop `else` = "no break happened" — ideal for search loops.
- Reach for `sum` / `any` / `all` / `next`/comprehensions first, and never modify a collection while iterating over it.